Write-up Challenge HackTheBox: ReactOOPs

Tác giả: Huỳnh Thanh Giảng

Ngày thực hiện: 25/03/2026

Nền tảng: HackTheBox

Danh mục: Web

Độ khó: Very Easy

Điểm: 10

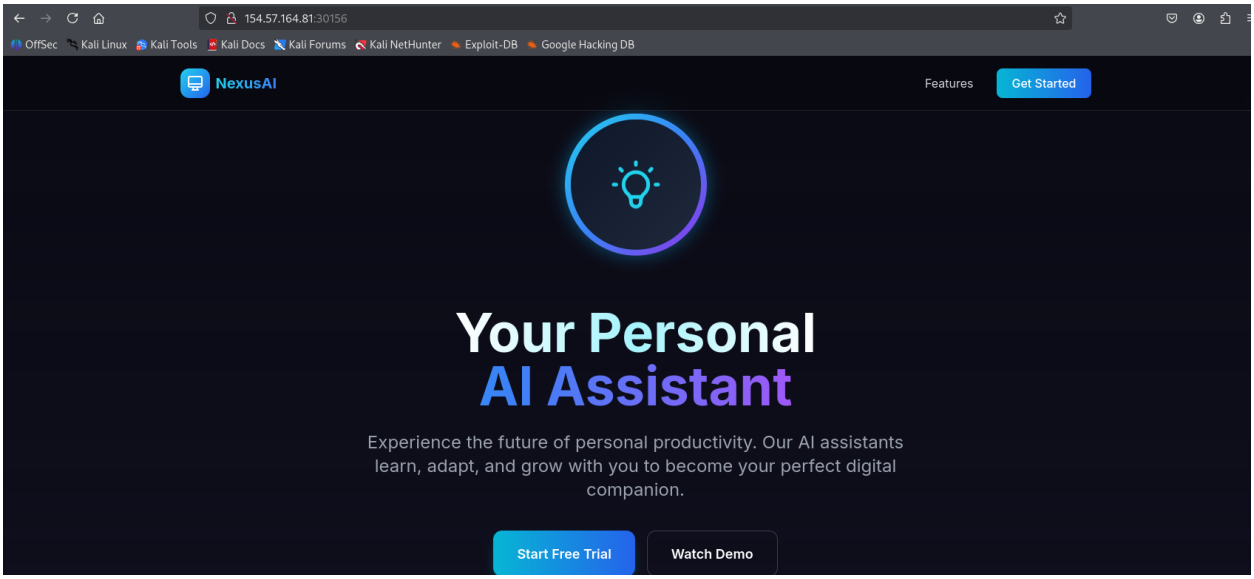
Link challenge: https://app.hackthebox.com/challenges/reactoops

1. Mô tả challenge & Khám phá source code ban đầu

Challenge Scenario

NexusAI's polished assistant interface promises adaptive learning and seamless interaction. But beneath its reactive front end, subtle glitches hint that user input may be shaping the system in unexpected ways. Explore the platform, trace the echoes in its reactive layer, and uncover the hidden flaw buried behind the UI.

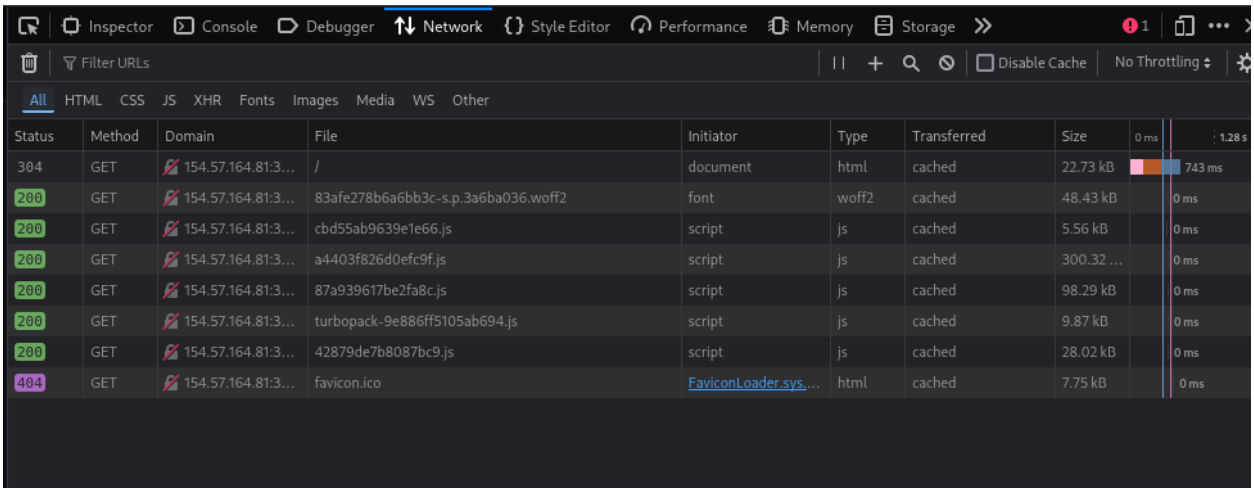
Challenge ReactOOPs là một web application thuộc danh mục Very Easy trên nền tảng HackTheBox. Khi spawn instance, ta được cung cấp một URL công khai (thường chạy trên port 80) với giao diện là một AI assistant mang tên NexusAI.



Để khám phá ban đầu, tôi mở **Chrome DevTools (F12)** và chuyển sang tab **Network**:

- Request đầu tiên (GET /) trả về một trang HTML (status 304 – from cache), kích thước khoảng 22.73 KB.
- Các request tiếp theo chủ yếu tải về các file JavaScript (các chunk từ turbopack/webpack), một file font .woff2, và một request 404 cho favicon.ico.

- Tất cả các tài nguyên JS đều được cached (status 200 hoặc 304).



Sau đó, tôi tải file ReactOOPS.zip về và giải nén để xem source code. Bên trong zip chứa các file và thư mục cấu hình của một dự án Next.js (package.json, thư mục app/, các file tsconfig.json, next.config.mjs…).

```
┌──(kali㉿huynhthanhgiang)-[~/Downloads]
└─$ unzip ReactOOPS.zip
Archive:  ReactOOPS.zip
   creating: web_reactoops/
[ReactOOPS.zip] web_reactoops/Dockerfile password:
  inflating: web_reactoops/Dockerfile
  inflating: web_reactoops/build-docker.sh
   creating: web_reactoops/challenge/
   creating: web_reactoops/challenge/app/
  inflating: web_reactoops/challenge/app/layout.tsx
  inflating: web_reactoops/challenge/app/page.tsx
  inflating: web_reactoops/challenge/app/globals.css
  inflating: web_reactoops/challenge/postcss.config.mjs
  inflating: web_reactoops/challenge/next.config.mjs
 extracting: web_reactoops/challenge/flag.txt
  inflating: web_reactoops/challenge/next-env.d.ts
  inflating: web_reactoops/challenge/tailwind.config.ts
  inflating: web_reactoops/challenge/package.json
  inflating: web_reactoops/challenge/tsconfig.json
```

Mở file package.json trong thư mục challenge/, ta thấy rõ các dependencies quan trọng:

```
"dependencies": {
  "next": "16.0.6",
  "react": "^19",
  "react-dom": "^19"
}
```

Điều này xác nhận challenge đang chạy Next.js 16.0.6 và React 19.

Kết luận: Challenge là một web application được xây dựng bằng Next.js. File zip được cung cấp giúp chúng ta có thể xem trực tiếp source code để phân tích sâu hơn (dựa trên cấu trúc thư mục app/, layout.tsx, page.tsx và các file config điển hình của Next.js). Từ tab Network ban đầu, ta thấy ứng dụng tải nhiều JavaScript chunks, cho thấy đây là một SPA (Single Page Application) hiện đại sử dụng React.

2. Phân tích lỗ hỏng và cơ chế Bypass

Lỗ hổng chính của challenge ReactOOPs là CVE-2025-55182, thường được gọi là React2Shell. Đây là một lỗ hổng cho phép thực thi lệnh từ xa (Remote Code Execution - RCE) mà không cần xác thực.

2.1. Công nghệ sử dụng và điểm yếu

```
"dependencies": {
  "next": "16.0.6",
  "react": "^19",
  "react-dom": "^19"
}
```

Từ file package.json ta thấy ứng dụng được xây dựng bằng:

- **Next.js 16.0.6**
- **React 19**

Next.js 16.0.6 sử dụng **React Server Components (RSC)** mặc định. RSC cho phép render component trên server và gửi về client dưới dạng **Flight Protocol** — một định dạng serialization đặc biệt của React.

Vấn đề nằm ở cách React 19 xử lý Flight payload. Khi deserializing (giải mã) dữ liệu, React thực hiện một số phép gán thuộc tính mà **không kiểm tra hasOwnProperty**. Cụ thể:

- React cho phép tồn tại các reference dạng $1:__proto__:then
- Nếu attacker kiểm soát được payload, có thể thực hiện **Prototype Pollution** lên Object.prototype.

2.2. Cơ chế Prototype Pollution dẫn đến RCE

Prototype Pollution xảy ra khi attacker có thể thay đổi thuộc tính của Object.prototype. Trong trường hợp này:

1. Attacker gửi một Flight payload đặc biệt chứa reference __proto__.
2. Khi React cố gắng resolve reference này (gọi .then()), nó vô tình chạy code mà attacker đã inject.
3. Code inject có thể là child_process.execSync() → cho phép thực thi lệnh hệ thống trên server.

2.3. Tại sao lỗ hỏng này bypass được dễ dàng ?

- Next.js 16.0.6 chưa có bản patch cho việc kiểm tra hasOwnProperty trong Flight deserializer.
- Ứng dụng không có WAF bảo vệ payload RSC.
- Không yêu cầu authentication → ai cũng có thể gửi payload độc hại.
- Source code trong app/page.tsx và layout.tsx sử dụng RSC mà không có thêm lớp bảo vệ nào.

3. Các bước khai thác thực tế

3.1. Chuẩn bị công cụ

Các bước chuẩn bị:

1. Truy cập GitHub repository: https://github.com/mrknow001/RSC_Detector
2. Tải về extension dưới dạng ZIP hoặc clone repository.
3. Mở Chrome → vào **chrome://extensions/**
4. Bật **Developer mode** (góc trên bên phải).
5. Nhấn **Load unpacked** và chọn thư mục chứa extens

3.2. Thực hiện khai thác

- Mở trình duyệt Chrome và truy cập địa chỉ challenge: **http://154.57.164.81:30156**
- Extension RSC Security Tool sẽ tự động chạy và phân tích trang web.

  - Nó phát hiện được việc trang đang sử dụng React Server Components qua fingerprint (window.__next_f và các chunk JavaScript).
  - Tab **RCE Exploit** sẽ tự động xuất hiện khi phát hiện lỗ hổng.

- Chuyển sang tab **RCE Exploit** trong extension.
- Trong phần Command, nhập lệnh sau:

```
cat /app/flag.txt
```

Kết quả:

RSC Security Tool

1. PASSIVE DETECTION

DETECTED

- Found: window.__next_f (App Router)

2. ACTIVE FINGERPRINT (RSC:1)

Start Fingerprint Probe

3. RCE EXPLOIT (CVE-2025-55182)

cat /app/flag.txt

EXEC

```
[+] Command: cat /app/flag.txt
[+] Output:
HTB{jus7_1n_c4s3_y0u_m1ss3d_r34ct2sh3ll___cr1t1c4l_un4uth3nt1c4t3d_RCE_1n_R34ct___CVE-2025-55182}
```